GEOQUEST AI

Geography Quiz Adventure

Project Specification & AI Build Prompt

An AI-powered, level-based geography game where players travel the world by answering increasingly difficult geography questions.

Prepared as a build-ready blueprint for AI coding agents (e.g. Antigravity, Cursor, Claude Code, Windsurf) and human engineers alike.

HOW TO USE THIS DOCUMENT

Section 1 is a ready-to-paste prompt for an AI coding agent. It instructs the agent to treat everything after it — the original project specification (Sections 2–8) plus supplementary implementation guidance we've added (Sections 9–13, each marked *Added*) — as its build reference. Hand the agent this entire PDF, or paste Section 1 followed by the rest as plain text.

Contents

1. AI Build Prompt

Copy-paste ready — give this to your coding agent

ROLE. You are an expert full-stack product engineer operating inside an agentic coding environment (e.g. Antigravity, Cursor, Claude Code, Windsurf). Build **GeoQuest AI — Geography Quiz Adventure** end to end, using this document as your single source of truth. Do not invent scope beyond it, and do not silently drop features.

PROJECT. GeoQuest AI is a level-based, AI-powered geography quiz game. Players progress through increasingly difficult levels (Countries → Capitals → Flags → Landmarks → … → Mixed / AI-Hard) by answering AI-generated, image-backed multiple-choice questions under a timer, earning XP, coins, lives, hints, achievements, and leaderboard rank.

SCOPE. Implement every item under "Core Features (1–25)", the schema in "Database Design", and the stack in "Recommended Tech Stack". Treat "Suggested Folder Structure", "Suggested REST API Endpoints", and "Non-Functional Requirements" as binding implementation guidance rather than suggestions. Treat "Optional Future Features" as explicitly out of scope for v1 — leave clean extension points but do not build them yet.

BUILD ORDER. Follow the phase order in "Development Roadmap & Milestones". Do not move on to animations, sound, or theming before the core gameplay loop — question → timer → answer → score → XP/coins → next question — works end to end against a real AI provider. After each phase, output a short summary of what was built, what was deferred, and any assumption you made where the spec was ambiguous, then continue.

AI QUESTIONS & IMAGES. Generate questions live from an LLM using the prompt contract in Core Feature 4 and the exact JSON shape in "Sample AI Question JSON Response" — the frontend must be able to consume it without guessing at field names. Never fall back to a static, hardcoded question bank; cache and de-duplicate server-side so a player never sees a repeat within a session. Resolve each question's image by searching Wikimedia Commons / Unsplash / Pexels with the AI-returned image keywords, per "Image Sources" — do not generate a fresh image per question.

GAME RULES ARE BALANCE-CRITICAL. Implement the XP formula, coin economy, lives, hint costs, boss-level cadence (every 5th level), and level-unlocking exactly as specified — these numbers define the game's feel and should not be approximated.

ENGINEERING CONSTRAINTS. TypeScript on both frontend and backend. All secrets (LLM API key, database URL, JWT secret, OAuth credentials, image API keys) come from environment variables, never hardcoded — ship a complete `.env.example`. Validate every API input with Zod. Hash passwords with bcrypt. Issue short-lived JWTs with refresh tokens. Build mobile-first and responsive from the start, and wire dark/light theming in from the first UI component rather than retrofitting it later.

WHEN THE SPEC IS AMBIGUOUS. Make the most game-appropriate assumption, state it explicitly in your output, and keep moving. Only stop to ask if a choice is genuinely irreversible (e.g. swapping the primary database).

DELIVERABLE. A working, runnable full-stack application — with a database seed script, `.env.example`, and a README covering local setup and deployment — that a reviewer can clone, configure, and run, matching every section that follows.

2. Project Overview

A Geography Quiz Game becomes far more engaging when it is designed like a progression-based game rather than a standard quiz app. Because questions are AI-generated, the game supports virtually unlimited content at increasing difficulty, rather than a fixed question bank that players eventually memorize.

GeoQuest AI is an AI-powered, level-based geography game where players travel around the world by answering increasingly difficult geography questions — unlocking new levels, topics, and rewards as they improve.

3. Core Features (1–25)

1. User Authentication

A professional login system with:

- Landing Page
- Login
- Register
- Forgot Password
- Email Verification (optional)
- Google Login
- Guest Mode
- Remember Me
- Logout

User profile includes:

- Name
- Avatar
- Email
- Country
- Current Level
- XP
- Coins
- Highest Score
- Total Games Played
- Accuracy %
- Rank

2. Dashboard

After login, the dashboard displays:

EXAMPLE — Dashboard

Welcome Prem!

Current Level: 12 | XP: 1350 | Coins: 420

Current Rank: Explorer | Accuracy: 81%

Plus quick actions:

- Play Button
- Leaderboard
- Achievements
- Daily Challenge
- Settings

3. Level-Based Gameplay

```
Level 1   Countries
   ↓
Level 2   Capitals
   ↓
Level 3   Flags
   ↓
Level 4   Landmarks
   ↓
Level 5   Mountains
   ↓
Level 6   Rivers
   ↓
Level 7   Oceans
   ↓
Level 8   Population
   ↓
Level 9   Climate
   ↓
Level 10  Mixed Geography
   ↓
Level 20+ Very Hard AI Questions
```

Difficulty increases automatically as players advance. See “Extended Level Map” (Section 9) for a suggested breakdown of levels 11–19.

4. AI-Generated Questions

Questions are generated dynamically using an AI API.

EXAMPLE PROMPT

```
Generate a Geography MCQ.
Difficulty: Medium
Topic: Capitals

Return JSON
{
  question,
  imagePrompt,
  options[],
  answer,
  explanation
}
```

- No repeated questions
- Unlimited gameplay

5. AI Image Support

Every question includes an image.

EXAMPLE — Landmark identification

Question: Which country is shown in this image?

Image: Himalayas

Options: India / Nepal / Bhutan / Pakistan

EXAMPLE — Landmark identification

Question: Which landmark is this?

Image: Eiffel Tower

Options: Paris / Rome / London / Berlin

Images can come from:

- Unsplash API
- Wikimedia Commons
- Pexels
- AI Image API

6. Timer

- Every question has 15 seconds or 30 seconds
- Countdown animation
- Running out of time counts as wrong

7. Four-Option MCQ

EXAMPLE — Multiple choice

What is the capital of Australia?

○ Sydney

○ Melbourne

● Canberra

○ Perth

Immediate feedback is shown after answering.

8. Real-Time Score

Always visible, with animated updates:

EXAMPLE — In-game score panel

Level 8 | Score: 140 | Correct: 14 | Wrong: 2

Coins: 80 | XP: 220 | Accuracy: 87%

9. XP System

- Correct answer: +20 XP
- Fast answer: +10 XP
- Perfect streak: +30 XP
- Wrong answer: −5 XP

10. Coins System

Earn coins by:

- Correct answers
- Daily login
- Completing levels
- Winning challenges

Coins can unlock:

- Themes
- Avatars
- Hint Packs
- Skip Question
- Double XP

11. Lives System

```
Start        ♥ ♥ ♥
Wrong answer ♥ ♥
Wrong        ♥
Wrong        Game Over
```

12. Hint System

Hints cost coins:

- **50-50** — remove two wrong answers
- **Reveal Hint** — e.g. “This country is in Europe.”
- **Skip Question**
- **Extra Time**

13. Level Unlocking

Players cannot access higher levels without clearing previous ones:

```
✓ Level 1
✓ Level 2
✓ Level 3
✗ Level 4  (locked)
```

14. Boss Levels

Every 5th level:

- 10 rapid-fire questions
- 20 seconds each
- Double XP
- Special Badge

15. Daily Challenge

Every day:

- 5 random AI questions
- Special rewards
- Leaderboard points

16. Weekly Tournament

Players compete globally. Top 100 earn:

- Badges
- Coins
- XP

17. Achievements

★ **World Traveler**

★ **Geography Master**

★ **Speed Demon**

★ **20 Correct Streak**

★ **100% Accuracy**

18. Leaderboard

- Global

- Country
- Friends
- Weekly
- Monthly
- All Time

19. Statistics

Charts showing:

- Accuracy
- Strongest Topics
- Weakest Topics
- Average Response Time
- Win Rate
- Games Played

20. Game Modes

- **Classic** — normal progression
- **Endless** — infinite AI-generated questions
- **Time Attack** — answer as many as possible in 2 minutes
- **Survival** — one wrong answer ends the game
- **Multiplayer (Future)** — two players compete live

21. Beautiful Animations

- Progress bar
- XP animation
- Coin animation
- Confetti on level completion
- Smooth page transitions
- Card flip effects
- Option hover effects
- Sound effects

22. Sound Effects

- Correct
- Wrong
- Countdown
- Victory
- Level Complete
- Coin Collection

A mute option is available.

23. Dark / Light Theme

Switch anytime.

24. Responsive Design

Works perfectly on:

- Desktop
- Tablet
- Mobile

25. Admin Panel

Admins can:

- View users
- Delete users
- Ban users
- View statistics
- Manage announcements
- Configure AI prompt templates
- View API usage
- Monitor leaderboards

4. Suggested Folder Structure

```
src/
  components/
  pages/
  hooks/
  services/
  api/
  context/
  assets/
  styles/
  utils/
  types/
```

5. Database Design

Suggested field types added for implementation clarity

Users

Field	Suggested Type	Notes
id	UUID / Serial	Primary key
name	String	
email	String (unique)	

Field	Suggested Type	Notes
password	String (hashed)	bcrypt hash; empty for OAuth-only accounts
avatar	String (URL)	
level	Integer	default 1
xp	Integer	default 0
coins	Integer	default 0
score	Integer	highest score achieved
rank	String	e.g. “Explorer”
accuracy	Float	percentage
createdAt	Timestamp	

Game History

Field	Suggested Type	Notes
id	UUID / Serial	Primary key
userId	UUID (FK → Users.id)	
level	Integer	
question	Text	
selectedAnswer	String	
correctAnswer	String	
score	Integer	
timeTaken	Integer (seconds)	
date	Timestamp	

Achievements

Field	Suggested Type	Notes
id	UUID / Serial	Primary key
userId	UUID (FK → Users.id)	
badge	String	
earnedAt	Timestamp	

Leaderboard

Field	Suggested Type	Notes
userId	UUID (FK → Users.id)	Primary key
score	Integer	
rank	Integer	
weeklyScore	Integer	
monthlyScore	Integer	

6. Recommended Tech Stack

Frontend

- React.js (with TypeScript)
- Vite (fast development / build)
- Tailwind CSS
- shadcn/ui (modern, accessible components)
- Framer Motion (animations)
- React Router
- React Query / TanStack Query (API state management)
- Zustand (lightweight global state)

Backend

- Node.js
- Express.js
- TypeScript
- JWT Authentication
- bcrypt (password hashing)
- Zod (validation)

Database

- PostgreSQL
- Prisma ORM

AI Integration

OpenAI API (GPT models) or Google Gemini API for generating geography questions in structured JSON, with:

- Difficulty
- Topic

- Four unique options
- Correct answer
- Short explanation
- Image search keywords

Image Sources

Instead of AI-generating every image (slower and costlier), use image search keywords returned by the AI and fetch images from:

- Wikimedia Commons
- Unsplash API
- Pexels API

This provides fast, relevant, and legally usable geography images.

Authentication

- JWT
- Google OAuth
- Refresh Tokens

Storage

- Cloudinary (avatars)
- Local storage (preferences)
- Redis (optional, for caching leaderboards and AI responses)

Deployment

- Frontend: Vercel
- Backend: Render or Railway
- Database: Neon PostgreSQL or Supabase PostgreSQL
- Object Storage (optional): Cloudinary

7. Overall Architecture

React + TypeScript + Vite

↓

Tailwind CSS + shadcn/ui + Framer Motion

↓

Node.js + Express (REST API)

↓

Backend Services & Integrations

- PostgreSQL (Prisma ORM)
- OpenAI / Gemini API (question generation)
- Image APIs (Wikimedia / Unsplash / Pexels)
- Redis (caching — optional)
- JWT + Google OAuth (authentication)

This stack is modern, scalable, developer-friendly, and well suited to a polished game experience. It supports AI-generated content, responsive design, real-time score updates, and user progression, and can later be extended with multiplayer, seasonal events, and richer game mechanics without a major architectural redesign.

8. Optional Future Features

★ Interactive world map where players click countries or landmarks
★ Adaptive AI that increases or decreases difficulty based on player performance
★ Voice-based questions and answers
★ Friend system with private challenges
★ Replay mode to review past questions with explanations
★ Progressive Web App (PWA) support for an app-like experience
★ Seasonal events and limited-time geography tournaments
★ Multi-language support
★ Personalized learning recommendations based on weak topics

ADDITIONAL IMPLEMENTATION GUIDANCE

The sections below (9–13) are not from the original brief — they are added to make the specification directly actionable for an AI coding agent or a human team, filling in the operational details (exact endpoints, sample payloads, phase order) the feature list implies but doesn't spell out.

9. Extended Level Map

Added

The brief specifies Levels 1–10 and names Level 20+ as very hard, AI-generated mixed questions. The bridge below (Levels 11–19) is a suggested extension consistent with the "boss level every 5th level" rule.

Level	Topic	Notes
1	Countries	
2	Capitals	
3	Flags	
4	Landmarks	
5	Mountains	Boss Level
6	Rivers	
7	Oceans	
8	Population	
9	Climate	
10	Mixed Geography	Boss Level
11–14	Advanced single-topic rounds (suggested)	Harder variants of Levels 1–9
15	Mixed + Timed	Boss Level
16–19	Cross-topic challenge rounds (suggested)	Two topics combined per question
20+	Very Hard, AI-Generated Mixed Questions	Boss Level every 5th level continues

10. Sample AI Question JSON Response

Added

A fleshed-out example matching the JSON contract in Core Feature 4, for the frontend and backend to agree on field names and types up front:

```json
{
  "id": "q_9f21ac",
  "difficulty": "medium",
  "topic": "capitals",
  "question": "What is the capital of Brazil?",
  "imagePrompt": "Brasilia Brazil capital city skyline",
  "options": ["Rio de Janeiro", "Sao Paulo", "Brasilia", "Salvador"],
  "answer": "Brasilia",
  "explanation": "Brasilia became Brazil's capital in 1960, replacing Rio de Janeiro.",
  "timeLimitSeconds": 20
}
```


11. Suggested REST API Endpoints

Added

Auth

Method	Path	Purpose
POST	/api/auth/register	Create account
POST	/api/auth/login	Email/password login
POST	/api/auth/google	Google OAuth login
POST	/api/auth/guest	Start a guest session
POST	/api/auth/refresh	Refresh an access token
POST	/api/auth/logout	Invalidate session

Profile & Dashboard

Method	Path	Purpose
GET	/api/users/me	Current profile, level, XP, coins, rank
PATCH	/api/users/me	Update profile / avatar / country
GET	/api/users/me/stats	Accuracy, strongest/weakest topics, history

Gameplay

Method	Path	Purpose
GET	/api/levels	Level list with lock/unlock status
POST	/api/questions/generate	Request a new AI-generated question
POST	/api/game/answer	Submit an answer; returns correctness + XP/coin delta
POST	/api/game/hint	Spend coins on a hint (50-50 / reveal / skip / extra time)
POST	/api/game/session/complete	Finalize a level/session and persist results

Social & Progression

Method	Path	Purpose
GET	/api/leaderboard	Global / country / friends / weekly / monthly / all-time
GET	/api/achievements	Achievements earned and available
GET	/api/challenges/daily	Today’s daily challenge
GET	/api/tournaments/weekly	Current weekly tournament standings

Admin

Method	Path	Purpose
GET	/api/admin/users	List / search users
DELETE	/api/admin/users/:id	Delete a user
POST	/api/admin/users/:id/ban	Ban a user
GET	/api/admin/stats	Platform-wide statistics
POST	/api/admin/announcements	Create/update an announcement
PUT	/api/admin/ai-prompt-templates	Edit AI question-generation prompt templates
GET	/api/admin/api-usage	AI/image API usage and cost tracking

12. Non-Functional Requirements

Added

Performance

- AI question generation should return in well under 3 seconds, with a loading state and a cached fallback pool if the AI provider is slow or unavailable
- Images should be lazy-loaded and cached (CDN or Redis) to avoid refetching the same landmark/flag repeatedly

Security

- Never trust client-submitted scores, XP, or coin totals — recompute and validate server-side on every answer submission
- Rate-limit AI question generation and auth endpoints per user/IP
- Sanitize and validate all AI-generated content before rendering it (the model is a data source, not a trusted template engine)

Scalability

- Design the question-generation service so it can be swapped or load-balanced across multiple AI providers without a frontend contract change
- Leaderboards should be computed incrementally or cached (e.g. via Redis sorted sets), not recalculated from full history on every request

Accessibility

- Color is never the only signal for correct/incorrect — pair it with icons/text
- All interactive elements are keyboard-navigable and screen-reader labeled
- Meets WCAG 2.1 AA color-contrast targets in both dark and light themes

Reliability & Testing

- Unit tests for XP/coin/scoring math and level-unlock logic
- Integration tests for the AI question contract (schema validation on every response)
- Graceful degradation: if the AI or image API fails mid-game, surface a retry state rather than crashing the session

13. Development Roadmap & Milestones

Added

Phase	Focus	Key Deliverables
1	Foundation	Project scaffold, auth (incl. guest mode), DB schema & migrations
2	Core Gameplay Loop	Levels 1–10, MCQ engine, timer, real-time score, XP/coins math
3	AI Question & Image Pipeline	Live AI generation, JSON contract, image lookup & caching, no-repeat logic
4	Progression Systems	Lives, hints, level unlocking, boss levels
5	Engagement Layer	Daily challenge, weekly tournament, achievements, leaderboards, statistics
6	Polish	Animations, sound effects, dark/light theme, full responsive pass
7	Admin & Launch Prep	Admin panel, deployment (Vercel/Render/Neon), monitoring, README

Optional Future Features (Section 8) are deliberately excluded from this roadmap and should only be scoped once v1 above is live and stable.